

Test Strategy Document

Prepaid By: *Kudakwashe Chris Chipangura*

1. Introduction

This Test Strategy outlines the overall testing approach for the QA Assessment project, which includes validating both the Laravel Backend API and the Flutter Mobile Application. The goal is to ensure product quality, reliability, and performance before release.

2. Testing Approach & Methodology

A hybrid approach combining manual testing, automation, and API testing will be used.

Manual Testing

- Focus on core user flows and UI validation for the Flutter app.
- Exploratory testing to uncover unexpected bugs.

Automation Testing

- Unit tests for backend logic (Laravel PHPUnit).

- Flutter widget tests for UI behaviour.
- API automated tests using Postman/Newman or REST Assured.

API Testing

- Validate endpoints for correctness, error handling, authentication, and performance.

Test Design Techniques

- Boundary Value Analysis (BVA)
 - Equivalence Partitioning
 - Error Guessing
 - State Transition Testing
-

3. Test Scope

In Scope

Backend API

- Authentication & Authorization
- CRUD Operations for all resources
- Data validation & error handling
- Business logic validation

- Response codes & payload structure
- Performance

Flutter Mobile App

- UI and UX validation
- API integration
- Navigation flow
- Data rendering
- Input handling and validation
- Offline handling

Out of Scope

- Security penetration testing
- Advanced load or stress testing

4. Test Levels

Unit Testing

- Laravel: Controllers, Services, Models.
- Flutter: Dart functions, widgets.

Integration Testing

- API database interactions.
- Flutter ↔ API communication.

System Testing

- End-to-end user scenarios.
- Full workflow validation.

User Acceptance Testing (UAT)

- Validate user stories.
- Check business requirements alignment.

5. Risk Analysis

Risk	Impact	Likelihood	Mitigation
API downtime	High	Medium	Early API mocks, retry test strategy
Flutter backend mismatch	High	High	Frequent sync with backend team
Device-specific bugs	Medium	High	Test on multiple devices/emulators
Data inconsistencies	High	Medium	Enforce DB seeding & cleanup

6. Entry & Exit Criteria

Entry Criteria

- Requirements are defined.
- Build is available (API and Flutter).
- Test environment set up.
- Access credentials and test data available.

Exit Criteria

- All planned test cases executed.
- Critical and high-severity bugs fixed.
- No blocker issues remain.
- Test summary report prepared.

7. Test Environment Requirements

Backend Environment

- Laravel app running with seeded database.
- Postman or API testing tool.

- Access to logs (Laravel logs, server logs).

Mobile App Environment

- Flutter setup (SDK installed).
- Android/iOS emulators or physical devices.
- Debug/app release build.

Tools

- GitHub for version control and Actions
 - Postman
 - Flutter DevTools
 - PHPUnit
 - VS Code
-

8. Resource Estimation

Testing Roles

- 1 QA Engineer performing manual & automation testing.

Effort Estimation

- Test strategy & planning: 4–6 hours

- Test case writing: 10–12 hours
 - Manual execution: 12–16 hours
 - API automation: 6–8 hours
 - Reporting & retesting: 4 hours
-

9. Deliverables

- Test Cases (Spreadsheet)
- Bug Reports
- Performance reports
- Test Summary Report
- Postman Json Exports both collection and enviroment